IMCS 3010U: Multiclass Image Classifier

By: Daniel Shklyarman

The Goal

Create a model that can take any number of images, under any number of classes, and learn to predict a given image's class. In this project, all training and testing is based on animal photos and either 3 or 4 classes.









Datasets and Tools

Training Data:

- Cats: Cats
- Dogs: Stanford Dogs
- Birds: Birds
- Fish: Fish
- Neither: Lions and Cheetahs, Cat V Rabbit, Animals, Kangaroo

Testing Data:

- Cats: Cat V Rabbit
- Dogs: Emotional Dogs
- Birds: Birds
- Fish: Fish
- Neither: Animals, Dolphin V Whale, Kangaroo

Tools and Environment:

- Keras, TensorFlow, and Pandas libraries for building, training, and testing the model
- Code written and ran in Jupyter for VSCode

Methods 1 - Preprocessing

- All images resized to 128x128 px using Keras built in image preprocessing
  - This is done in order to make the model a bit faster to train
- Each image is normalized by dividing each pixel by 255
- Images are augmented using Keras image preprocessing in order to avoid overfitting
  - Rotation: 40 degrees
  - Shear: 0.2
  - Zoom: 0.2
  - Horizontal Flip
  - Fill: Nearest. Means that empty pixels are filled in with the value of the closest known pixel

Methods 2 - CNN and Pooling

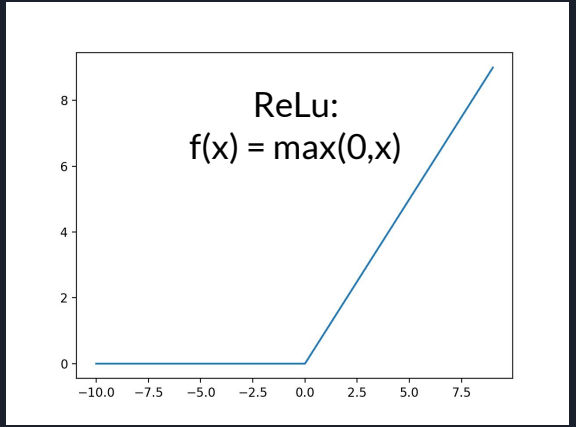
- Convolutional Neural Networks use filters (or kernels) of varying size to analyze and extract features from the images in the dataset
  - Used primarily to prevent overfitting and extracting higher level features that cannot be found via just looking at one pixel at a time
    - I.e. edges
  - In this model, all convolutional layers use a basic 3x3x3 filter for simplicity
- Pooling layers drag a ‘tile’ of a given size over a matrix and applying a function to all the numbers within the tile. A new matrix is then created from the numbers given by dragging the tile over the entirety of the original matrix
  - In this model, the function is simply to keep the largest number in the tile (MaxPool). This serves to keep important parts of the data and reduce noise

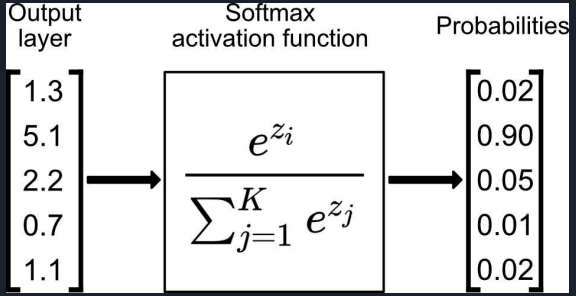
Methods 3 - Dense and Dropout Layers

- Dense layers are made up of a set number of ‘neurons’ that each receive input from all of the neurons in the previous layer and their outputs are fed to each neuron of the next layer
  - Only one Dense hidden layer in my models because too many makes the model prone to overfitting
  - Final layer is a Dense layer since the Softmax activation requires a 1D array
- Dropout layers are a regularization technique to help prevent overfitting by randomly dropping some amount of the neurons (setting their output to 0) during training - 20% in this case. This prevents the model from becoming overly reliant on the output of specific neurons.

Methods 4 - Activation, Loss, and Optimization Functions

- ReLu is used as the activation function at all intermediate layers as it is the most generally effective activation function
  - Also relatively simple computationally
- Softmax is used at the final layer in order to take the output of the CNN and turn it into a probability distribution, with the highest probability being the predicted class
- Categorical Cross Entropy was used as the loss function, as it works for NNs where the output is a probability distribution and there is only one possible class the input can belong to
- ADAM is used as the optimization function as it is generally the most effective, and memory was not a concern when training the models





$$\text{Loss} = -\sum_{i=1}^{\text{output size}} y_i \cdot \log \hat{y}_i$$

Methods 5 - Model Summary

1. Convolutional layer of 16 filters (3x3) and ReLu activation
2. Max Pool (2x2)
3. Convolutional layer of 32 filters (3x3) and ReLu activation
4. Max Pool (2x2)
5. Convolutional layer of 64 filters (3x3) and ReLu activation
6. Max Pool (2x2)
7. Flatten
8. Dense Layer of 512 nodes and ReLu activation
9. Dropout (0.2)
10. Dense Layer of as many nodes as there are classes and Softmax activation

Methods 6 - Epochs, Batchsize, and Train/Validation split

- The Cat-Dog-Neither model was trained with 20 epochs, however all the other models were trained with 15 due to time constraints
- Batch size was arbitrarily set to 512
- 20% of the training data was used for validation

Testing

- Testing Images were split off from the training datasets, or others containing pictures of the same animal
- Each testing set had 100 images
- Images were all resized to 128x128 and normalized so that all pixel values were between 0 and 1
    - Normalization is especially important as accuracy is essentially 0% without it
- Accuracy for all photos was calculated by feeding the testing images to the model, and dividing the number of correct classifications by the number of testing images - 100 in this case

Results

Cat-Dog-Neither Model

1. Cat: 0.80
2. Dog: 0.44
3. Neither: 0.61

Cat-Dog-Fish Model

1. Cat: 0.7
2. Dog: 0.32
3. Fish: 0.89

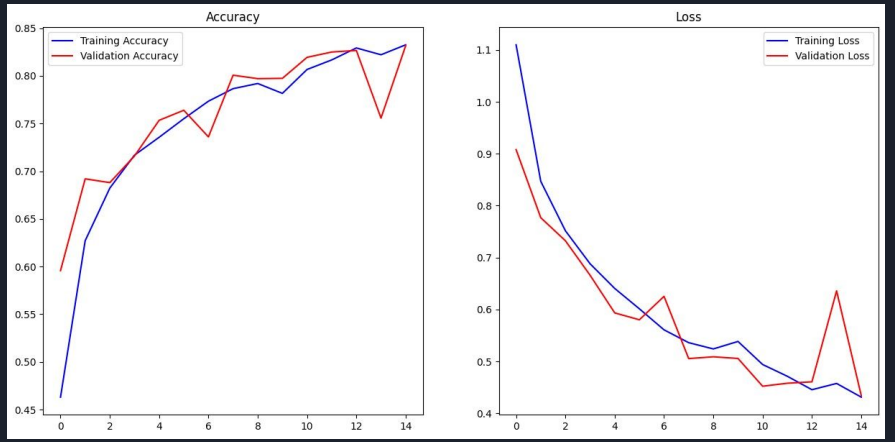
Cat-Dog-Bird-Fish Model

1. Cat: 0.85
2. Dog: 0.27
3. Bird: 0.76
4. Fish: 0.77

Cat-Bird-Fish Model

1. Cat: 0.85
2. Bird: 0.74
3. Fish: 0.85

Accuracy and Loss for CBF Model



Conclusions

1. The models are consistently far worse at identifying dogs than they are at identifying other animals
    a. This is likely due to the dog photos being more varied, where the other animal photos usually focus on the animal

Future Work 1 - Model Adjustment

- Optimize the model by varying various hyper-parameters
  - Change number of layers
  - Change number and size of filters in Convolutional layers
  - Change activation functions
  - Try different dropout and batch normalization layers
- Try other training sets, especially for dogs

Future Work 2 - Expansion

- Expand model beyond classification
  - Locate animal in photo
  - Identify multiple animals in a single photo
- Test on other types of classes - i.e. landscapes
  - Mix multiple types of classes - i.e. landscapes, animals, appliances
  - See if ‘Other’ can be made to work as a class

Future Work 3 - Challenges and Limitations

- Models take time to train (~2.5 mins per epoch at current model size/complexity), and with so many hyper-parameters to test, the time required to test all of the ideas becomes impractical
  - This especially the case with training and testing the model on more than 4 classes
  - Jupyter is far faster and more reliable than Colab, so some smaller ideas are practical - i.e. changing activation functions and trying out various dropout and batch normalization layers